



SC2002 - SCED - Group 2 - Report

APPENDIX B:

Declaration of Original Work for SC2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

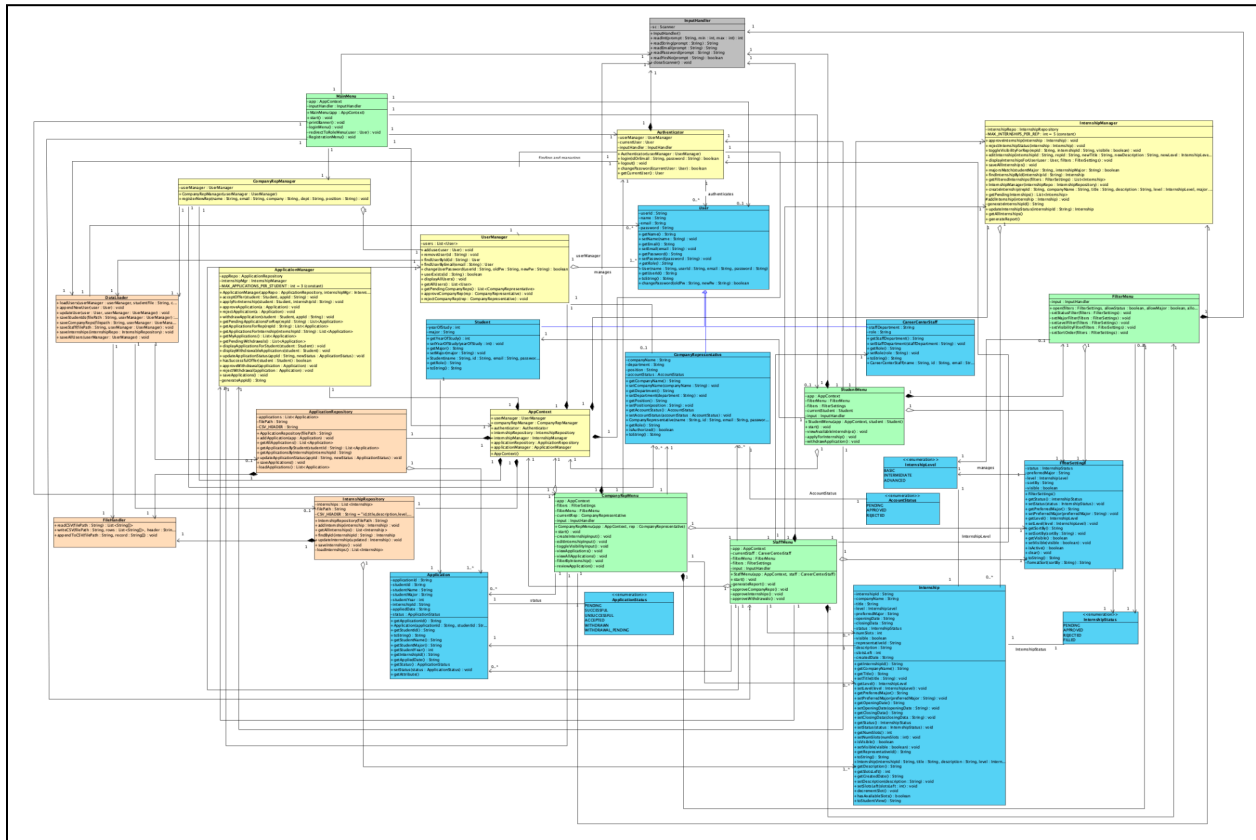
We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course	Lab Group	Signature / Date
VIVEK SHREY	SC2002	SCED	 15-11-2025
KUTTIYIL RAJATH KRISHNA	SC2002	SCED	 15-11-2025
MANI ARVIN AROCKIARAJ	SC2002	SCED	 15-11-2025
VARUN KRISHNA	SC2002	SCED	 15-11-2025
T MARUTHUVEL	SC2002	SCED	 15-11-2025

Important notes:

1. Name must **EXACTLY MATCH** the one printed on your Matriculation Card.
2. Student Code of Academic Conduct includes the latest guidelines on usage of Generative AI and any other guidelines as released by NTU.

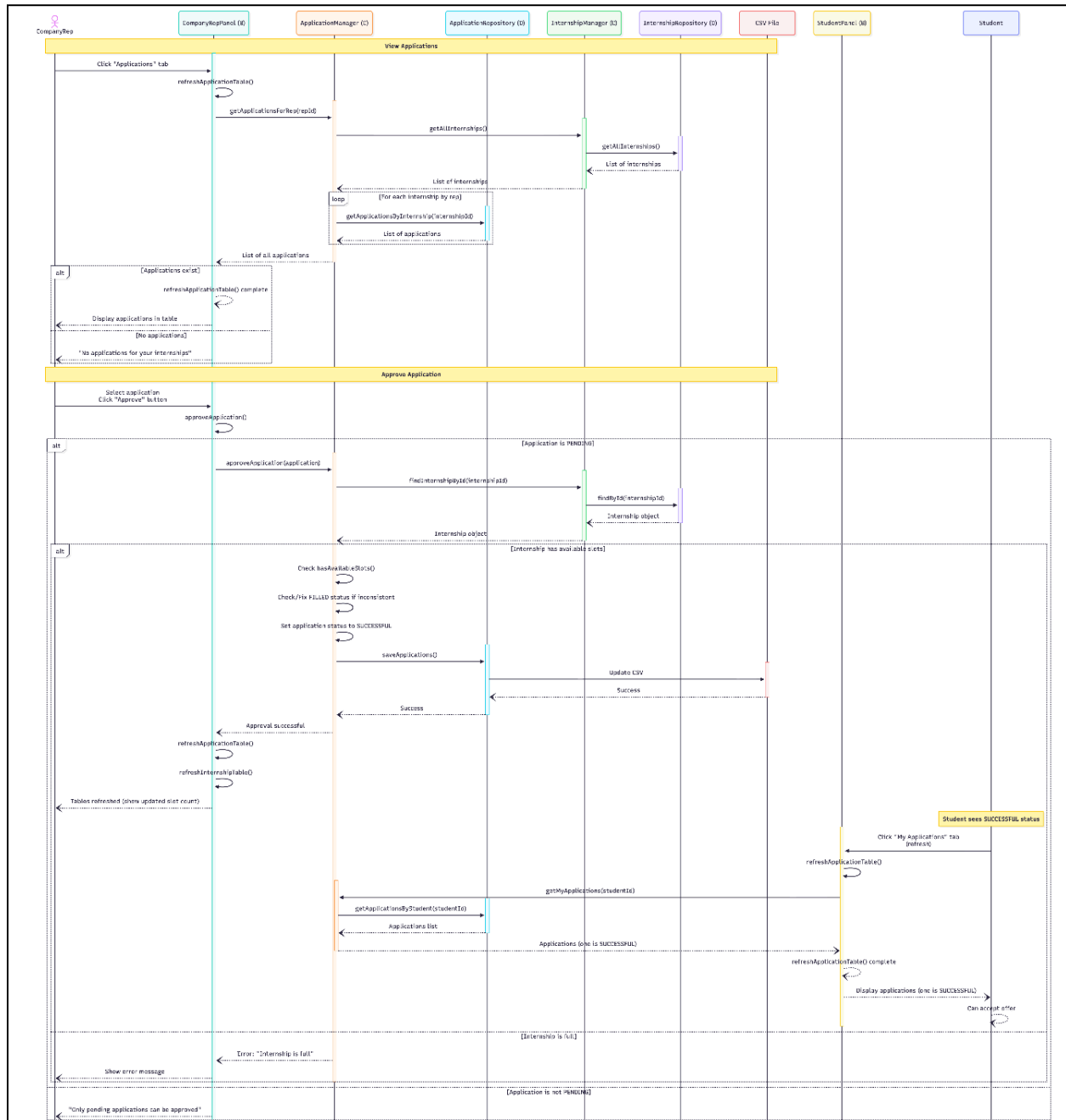
A. Class Diagram



Notes: (uploaded clear image in GitHub)

1. Workflow boundaries mirror real operational approvals. Each approval handoff (account approval, opportunity approval, application approval, withdrawal approval) is reflected as a dedicated control class, ensuring the diagram maps *exactly* to the real multi-stage decision process.
2. All state transitions are modeled as domain events, not adhoc flags. Cascaded transitions (eg. opportunity slot confirmation can only occur after CompanyRep approval) are not encoded randomly; the relationships in the diagram explicitly help in the flow of events.
3. Instead of scattering reads/writes everywhere, the diagram centralizes persistence as a cohesive module (DataLoader, File Handler) that abstracts file structures entirely.
4. The diagram supports large numbers of students, companies, and opportunities by avoiding the idea of just one controller. Each workflow controller manages its own domain slice, allowing the system to scale if this became a real admin tool.

B. Sequence Diagram



Notes: (uploaded complete image in GitHub, and two more sequence diagrams as well.)

1. Cascading state logic is shown clearly: approval triggers a multi-step chain (ApplicationManager → InternshipManager → Internship), not just a single update.
2. Acceptance workflow ensures exclusivity, updating the selected application and resolving all other applications for that student before finalizing internship slots.

3. Repositories remain passive data stores, with all decisions handled by the respective Managers to maintain clean domain layering.
4. AppContext centralizes dependency resolution, reducing coupling and keeping menus free of direct subsystem references. Responsibilities are correctly routed to the respective components that execute their functionalities - managers handle rules, repos handle data, boundaries handle input.

C. Additional Features

- We implemented a Java Swing GUI on top of the required CLI. The GUI introduces frames, buttons, panels, tables, and modal dialogs for a much more intuitive interaction flow. It integrates cleanly with our existing BCE architecture by acting as an alternative Boundary layer without changing any core logic. This hybrid design enhances usability, reduces input errors, provides real-time visual updates, and demonstrates that the system can support multiple UI front-ends. It can also be run in CLI mode.

```
package main.gui;

import main.control.AppContext;
import main.entity.*;

import javax.swing.*;
import java.awt.*;

/**
 * Main GUI entry (Boundary) for the Internship Placement Management System.
 * Hosts role-specific panels and routes actions to the Control layer via AppContext.
 */
public class MainGUI extends JFrame {
    private final AppContext app;
    private CardLayout cardLayout;
    private JPanel mainPanel;
    private LoginPanel loginPanel;
    private RegistrationPanel registrationPanel;
    private StudentPanel studentPanel;
    private CompanyRepPanel companyRepPanel;
    private StaffPanel staffPanel;
    private User currentUser;

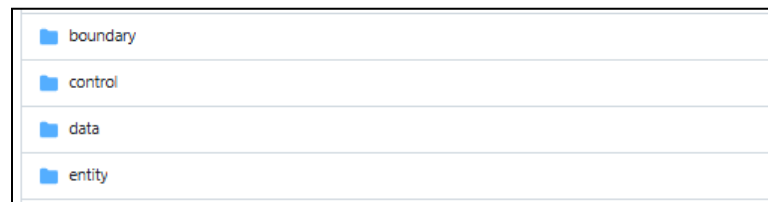
    public MainGUI(AppContext app) {
        this.app = app;
        initializeGUI();
    }
}
```

Supporting Figure 1: Java Swing GUI

- When a student applies for an internship, it can no longer be seen in available internships (to avoid duplicate ones being created). If they withdraw successfully, then it will reappear in the available list for them to apply again.
- Slots Left updated only when a student successfully accepts an offer, and other students cannot parallelly accept, even if their application was approved (first come, first serve).
- All Data is saved in real time to the CSV files, and stays consistent for all users, sessions.

D. Design Considerations

1. **Overall Design Approach:** We structured the Internship Placement System around a clear **Boundary, Control, Entity (BCE)** architecture. Boundary classes (e.g. the various Menu and Swing screens) handle user interaction only, Control classes (managers, authenticators) contain the logic/workflows, and Entity/Repository classes handle domain data and persistence.



Supporting Figure 2: BCE Architecture in use

2. **OO Concepts Applied:**

- **Encapsulation** - All entity classes (User, Student, Internship, Application, etc.) maintain private fields with controlled access through getters and validated setters.

```
public class Internship {
    private String internshipId;
    private String title;
    private String description;
    private InternshipLevel level;
    private String preferredMajor;
    private String openingDate;
    private String closingDate;
    private InternshipStatus status;
    private String companyName;
    private String representativeId;
    private int numSlots;    // total slots
    private int slotsLeft;  // available slots
    private boolean visible;
    private String createdAt;
```

Supporting Figure 3: Internship's private fields

```
//Getters & Setters
public String getInternshipId() { return internshipId; }
public String getTitle() { return title; }
public String getDescription() { return description; }
public InternshipLevel getLevel() { return level; }
public String getPreferredMajor() { return preferredMajor; }
public String getOpeningDate() { return openingDate; }
public String getClosingDate() { return closingDate; }
public InternshipStatus getStatus() { return status; }
public String getCompanyName() { return companyName; }
public String getRepresentativeId() { return representativeId; }
public int getNumSlots() { return numSlots; }
public int getSlotsLeft() { return slotsLeft; }
public String getCreatedDate(){return createdDate;}
public boolean isVisible() { return visible; }
```

Supporting Figure 4: Internship's Getters & Setters

- **Inheritance** - A general User superclass captures shared attributes (ID, name, password) and behaviours. Specific roles (Student, CompanyRepresentative, CareerCenterStaff) extend this base class with additional fields and methods relevant to their responsibilities. This reduces duplication while keeping role-specific behaviour cleanly isolated.

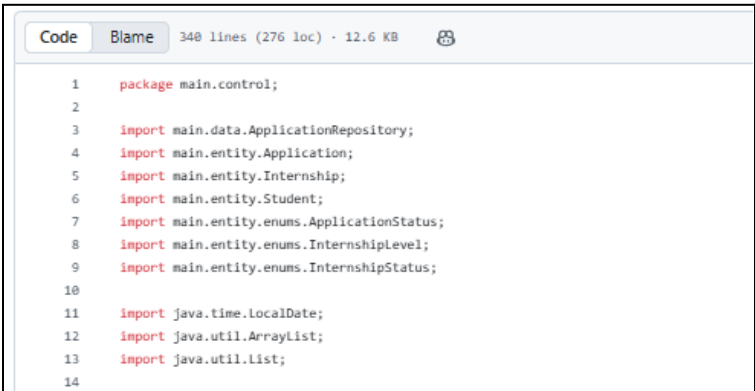
```
1 package main.entity;
2
3 import main.control.UserManager;
4 import main.data.DataLoader;
5
6 /**
7  * Represents a Student user.
8  *
9  * OOP PRINCIPLES APPLIED:
10 * 1. INHERITANCE - Extends User class, inherits common properties
11 * 2. SINGLE RESPONSIBILITY - Only handles student-specific data
12 * 3. LISKOV SUBSTITUTION PRINCIPLE - Can be used anywhere a User is expected
13 */
14
15 public class Student extends User {
16     private int yearOfStudy;
17     private String major;
18
19     public Student(String name, String id, String email, String password, int yearOfStudy, String major) {
20         super(name, id, email, password);
21         this.yearOfStudy = yearOfStudy;
22         this.major = major;
23     }
24 }
```

Supporting Figure 5: Student entity inheriting from user

- **Polymorphism** - Control classes interact with the abstract User type, while runtime behaviour depends on the underlying subclass. For example, menus and managers decide available actions based on whether the actual instance is a Student, CompanyRepresentative, or CareerCenterStaff. This supports adding new user types without modifying existing logic.
- **Abstraction** - The design hides low-level file I/O and entity storage details inside repositories (ApplicationRepository, InternshipRepository). Boundary classes and managers interact only with high-level operations (e.g., “get

applicants”, “set status”), not raw file handling. This makes the system easier to modify and reason about.

- **Composition** - Complex workflow objects (e.g., `ApplicationManager`) internally use repositories and other managers to handle multi-step processes. These components are combined meaningfully but maintain clear ownership and lifecycle boundaries.



The screenshot shows a code editor window with a tab labeled 'Code'. The editor displays the following Java code:

```
1 package main.control;
2
3 import main.data.ApplicationRepository;
4 import main.entity.Application;
5 import main.entity.Internship;
6 import main.entity.Student;
7 import main.entity.enums.ApplicationStatus;
8 import main.entity.enums.InternshipLevel;
9 import main.entity.enums.InternshipStatus;
10
11 import java.time.LocalDate;
12 import java.util.ArrayList;
13 import java.util.List;
14
```

Supporting Figure 6: ApplicationManager using other repositories and other managers

3. SOLID Principles Reflected in the Design:

- **Single Responsibility Principle (SRP)** - The `Application` class is a direct application of SRP because it is responsible only for representing application data. It performs just ONE job: storing and providing access to student application information (student details, internship ID, status, etc.).

<pre>public class Application { private String applicationId; private String studentId; private String studentName; private String studentMajor; private int studentYear; private String internshipId; private String appliedDate; private ApplicationStatus status; }</pre>	<i>Supporting Figure 7: The <code>Application</code> entity follows SRP by encapsulating only application-related data (IDs, student info, internship link, status, and date). All workflow logic like applying, approving, accepting offers, and withdrawals, is handled exclusively by other classes.</i>
--	--

- **Open–Closed Principle (OCP)** - The `User` superclass stays unchanged while other roles (generalization) like `Student`, `CompanyRepresentative`, and `CareerCenterStaff` extend it with their own fields and behaviours. New roles, like `FacultyManager` can be added simply by creating another subclass, without modifying existing code. This keeps the system open to extension but closed to modification.
- **Liskov Substitution Principle (LSP)** - The set of classes including - `Student`, `CompanyRepresentative`, and `CareerCenterStaff` can all be used wherever a `User` is expected, because the base class contains only shared behaviour. Role-specific logic stays in the subclasses, so substitutability is never broken and managers can operate on the abstract `User` type safely.
- **Interface Segregation Principle (ISP)** - ISP wasn't needed because the codebase uses concrete classes and inheritance, not interfaces. Classes depend directly on the specific methods they use (e.g., `UserManager` uses `User` methods, `InternshipRepository` uses `Internship` methods), so there are no "fat interfaces" forcing unused methods. The direct dependency model already achieves ISP's goal: classes only depend on what they need, without the overhead of interface abstractions for a project of this size.
- **Dependency Inversion Principle (DIP)** - Dependencies are injected through `AppContext` rather than created by classes. Menus and managers receive their dependencies (managers, repositories) via `AppContext`, keeping high-level classes independent of how dependencies are instantiated. This enables easier testing and future changes without requiring interface abstractions for a project of this size.

4. Extensibility & Maintainability

Features	Description
Pluggable UI (CLI + Swing)	Boundary classes stay thin, allowing the GUI to be added without modifying core logic.
Centralised Context	<code>ApplicationContext</code> supplies all dependencies, reducing coupling and simplifying future changes.
Clear Logic Mapping	All system rules are placed in their respective managers, making updates easy and predictable.
File I/O Abstraction	Repositories isolate storage details, enabling painless migration to databases later.

5. Design Patterns & Trade Offs

Aspect	Description
BCE Layering	Separates UI, logic, and data cleanly at the cost of more classes.
Repository Pattern	Abstracts storage access, trading extra indirection for easier extensibility.
<code>ApplicationContext</code> as Service Locator	Centralises object creation, simplifying wiring with a small global-like trade-off.
Manager Orchestrated Workflows	Complex processes stay in managers, making logic clearer but managers slightly larger.

6. Alternative Patterns Considered

Avoided	Reason
Fat Entities	Rejected to avoid mixing domain state with multi-entity workflows.
Direct CLI-Only Design	Avoided because it blocks future UI extensions and hurts maintainability.
Observer / Full DI	Considered unnecessary for the system's scale and would overcomplicate the design.

E. Reflection

This assignment made the gap between “knowing” design principles and actually “using” them very clear. We took architecture very seriously because the biggest challenge was coordinating multi-step workflows (approvals, slot updates, offer acceptance) without letting logic break across classes. We decided to code the rules into managers and keep entities passive, giving the system a stable backbone. Through that, we realized the importance of following OOP principles (especially SOLID - like Single Responsibility) and how clean layering matters more in practice than on slides.

Another key learning point was how much easier extensibility becomes when boundaries stay thin. Because control logic was entirely organized in managers, adding the GUI later was trivial for us. That made us understand the importance of the Open - Closed Principle.

`AppContext` proved to be very useful in dependency management. Centralizing construction simplified the code to a great extent and showed us that even a small-scale system benefits from mild inversion of control, without doing too much!

In order to improve the system, we hope to focus on stronger automated testing around managers, and possibly introduce small interfaces for repositories to make future storage changes even smoother. We also hope to deploy this in future and allow students in Singapore to use our portal for their internship applications. Overall, the project taught us how good design reduces cognitive load, makes changes predictable, and prevents architecture from collapsing under its own weight.

F. GitHub Repository

Click here: https://shreyvivek.github.io/oops_internship_system/